

```

1334     boxes: pd.DataFrame,
1335     stops: list[dict[str, Any]],
1336     start_s: float,
1337 ) -> dict[str, Any] | None:
1338     """Evaluate one multi-stop transport sortie with decreasing payload."""
1339
1340     indexed = boxes.set_index("货箱编号", drop=False)
1341     all_ids = [box_id for stop in stops for box_id in stop["box_ids"]]
1342     batch = indexed.loc[all_ids]
1343     total_mass = float(batch["单箱质量 (kg)"].sum())
1344     total_volume = float(batch["单箱体积 (m³)"].sum())
1345     if total_mass > g.payload_kg + 1e-9 or total_volume > g.volume_m3 + 1e-12:
1346         return None
1347
1348     takeoff_s = start_s + g.prep_s + len(all_ids) * g.load_per_box_s
1349     current_node = depot
1350     current_time = takeoff_s
1351     remaining_payload = total_mass
1352     energy = 0.0
1353     completions: list[dict[str, Any]] = []
1354     leg_records: list[dict[str, Any]] = []
1355     service_records: list[dict[str, Any]] = []
1356
1357     for stop in stops:
1358         service_id = str(stop["service_id"])
1359         service = nodes[service_id]
1360         leg = leg_metrics(dem, current_node, service, g, remaining_payload)
1361         arrival_s = current_time + leg["time_s"]
1362         leg_records.append(
1363             {
1364                 "origin": current_node.node_id,
1365                 "destination": service_id,
1366                 "start_s": current_time,
1367                 "end_s": arrival_s,
1368                 "payload_kg": remaining_payload,
1369                 **leg,
1370             }
1371         )
1372         energy += leg["energy_kwh"]
1373         stop_rows = indexed.loc[list(stop["box_ids"])].copy().reset_index(drop=True)
1374         stop_rows["_hard"] = stop_rows.apply(box_deadline, axis=1)
1375         stop_rows["_hard_sort"] = [float(x) if x is not None else float("inf") for x in stop_rows["_hard"]]
1376         stop_rows = stop_rows.sort_values(
1377             ["_hard_sort", "应急优先系数", "货箱编号"], ascending=[True, False, True]
1378         )
1379         handover_start = arrival_s + g.base_handover_s
1380         for order, (_, row) in enumerate(stop_rows.iterrows(), start=1):
1381             completion_s = handover_start + order * g.handover_per_box_s
1382             hard = box_deadline(row)
1383             expected = float(row["期望送达时间 (s)"])
1384             completions.append(
1385                 {
1386                     "box_id": str(row["货箱编号"]),
1387                     "service_id": service_id,
1388                     "completion_s": completion_s,
1389                     "hard_deadline_s": hard,
1390                     "hard_on_time": hard is None or completion_s <= hard + 1e-9,
1391                     "expected_s": expected,
1392                     "priority": float(row["应急优先系数"]),
1393                     "tardiness_s": max(0.0, completion_s - expected),
1394                 }
1395             )
1396         depart_s = arrival_s + g.base_handover_s + len(stop_rows) * g.handover_per_box_s
1397         service_records.append(
1398             {"service_id": service_id, "arrival_s": arrival_s, "depart_s": depart_s}
1399         )
1400         remaining_payload -= float(stop_rows["单箱质量 (kg)"].sum())
1401         current_node = service
1402         current_time = depart_s
1403
1404     back = leg_metrics(dem, current_node, depot, g, 0.0)
1405     return_s = current_time + back["time_s"]
1406     leg_records.append(
1407         {
1408             "origin": current_node.node_id,
1409             "destination": depot.node_id,
1410             "start_s": current_time,

```